

Introduction to AWS Lambda

AWS Lambda is a serverless, event-driven compute service provided by AWS that allows us to run application code without provisioning, managing, or maintaining servers.

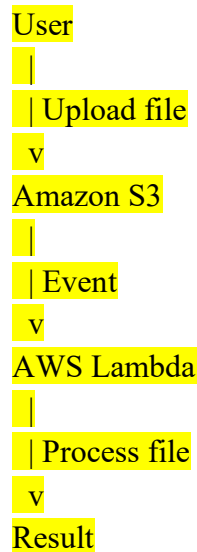
In traditional application hosting, we normally need to create a server such as an EC2 instance, install the operating system and required software, deploy the application, monitor the server, apply patches, and manage scaling.

With Lambda, AWS manages the underlying infrastructure for us. We only need to provide our application code and configure how the function should execute.

Simple definition

AWS Lambda allows us to run code in response to events without managing the underlying servers.

For example, suppose a user uploads a file to an S3 bucket:



The Lambda function can automatically execute when the file is uploaded.

What is Serverless Computing?

The term serverless can sometimes be confusing.

Serverless does not mean that there are no servers.

Servers are still required to execute our applications, but AWS manages those servers for us.

Traditional approach — EC2

Suppose we want to run a Python application.

We might do:

Create EC2



Choose AMI



Configure instance



Connect using SSH



Install Python



Install dependencies



Deploy application



Monitor server



Patch OS



Manage scaling

We are responsible for much of the infrastructure.

Serverless approach — Lambda

With Lambda:

Write application code



Create Lambda function



Select runtime



Deploy code



AWS executes the code

AWS takes care of the underlying infrastructure.

Therefore:

Serverless shifts infrastructure management from the customer to AWS.

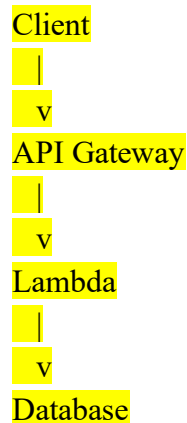
Why Do We Use AWS Lambda?

Lambda is useful when we want to execute code without maintaining continuously running servers.

Common use cases include:

1. API backends

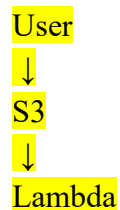
Lambda can act as the backend for an API.



For example, when a user requests account information, API Gateway can invoke Lambda and Lambda can retrieve the required data.

2. File processing

Lambda can process files uploaded to S3.





Process file

Example:

A user uploads an image.

Lambda could:

- Resize the image
- Validate the file
- Extract metadata
- Generate thumbnails
- Move the processed file

3. Scheduled tasks

Lambda can execute code according to a schedule.

For example:

EventBridge



Every day at 12 AM



Lambda



Perform task

It could be used for periodic automation or maintenance tasks.

4. AWS automation

Lambda can also be used to automate AWS operations.

For example:

CloudWatch/EventBridge



Lambda



AWS API



Perform operation

A Lambda function could interact with other AWS services through AWS APIs.

1.4 Event-Driven Architecture

One of the most important concepts in Lambda is event-driven execution.

An event is something that happens and causes Lambda to execute.

For example:

S3 file uploaded



Event



Lambda executes

Another example:

HTTP request



API Gateway



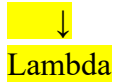
Lambda

Another:

Scheduled time



EventBridge



So the basic concept is:

Something happens → an event is generated → Lambda is invoked → Lambda executes the code → Lambda returns a result or performs an action.

1.5 Lambda Function

A Lambda function is the basic unit of execution in AWS Lambda.

It contains the code that AWS executes.

For example, a simple Python Lambda function could be:

```
def lambda_handler(event, context):  
    return {  
        "statusCode": 200,  
        "body": "Hello from AWS Lambda!"  
    }
```

Here:

```
lambda_handler  
|  
+---- event  
|  
+---- context  
|  
+---- response
```

The function receives input and performs an operation.

1.6 Runtime

A runtime provides the environment required to execute our Lambda code.

Examples include runtimes for:

- Python

- Node.js
- Java
- .NET
- Ruby

For our hands-on activity, we can use Python because it is simple for learning Lambda fundamentals.

For example:

Lambda Function

```
|  
+---- Runtime: Python  
|  
+---- Handler  
|  
+---- Application Code
```

The runtime determines how AWS executes the code.

Example

If we select:

Runtime = Python

AWS provides the required Python execution environment for the function.

We don't need to install Python on an EC2 server ourselves.

1.7 Handler

The handler tells Lambda which function in our code should be executed.

Suppose our Python file is:

lambda_function.py

and inside it we have:

```
def lambda_handler(event, context):  
    return "Hello"
```

The handler is:

lambda_function.lambda_handler

This means:

```
lambda_function  
|  
+---- lambda_handler()
```

The first part is the Python file/module.

The second part is the function that Lambda should execute.

Easy way to remember

Handler = Entry point of the Lambda function.

When Lambda receives an invocation, it knows where execution should begin because of the handler configuration.

1.8 Event

An event is the input provided to a Lambda function when it is invoked.

For example:

```
{  
  "name": "Asif"  
}
```

Lambda receives this event.

Our Python code can access it:

```
def lambda_handler(event, context):  
    name = event["name"]  
  
    return {
```

```
"statusCode": 200,  
"body": "Hello " + name  
}
```

The event structure depends on what triggered Lambda.

For example:

API Gateway event

HTTP Request



API Gateway



Lambda

The Lambda event contains information related to the HTTP request.

S3 event

File uploaded



S3 event



Lambda

The event contains information about the S3 bucket and object.

So:

Event = Input information received by Lambda during invocation.

1.9 Context

Lambda also provides a context object to the handler.

Example:

```
def lambda_handler(event, context):  
    print(context)
```

The context provides information about the current execution environment, such as:

- Function name
- Function version
- Request ID
- Remaining execution time

For example:

```
def lambda_handler(event, context):
```

```
    print("Function:", context.function_name)
```

```
    return {
        "statusCode": 200,
        "body": "Lambda executed successfully"
    }
```

The context is useful when troubleshooting and understanding the current Lambda execution.

1.10 Trigger

A trigger is a configuration that causes Lambda to be invoked when a specific event occurs.

Examples include:

API Gateway

S3

EventBridge

SQS

SNS

CloudWatch

For example:

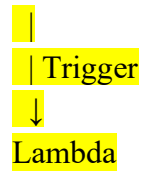
User

|

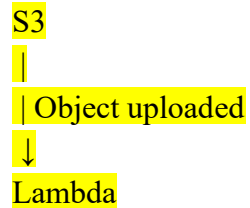
| HTTP request

↓

API Gateway



Or:



Important distinction:

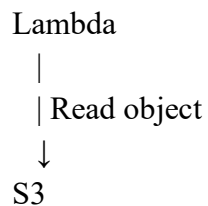
Event = information/input sent to Lambda.

Trigger = mechanism/service configured to invoke Lambda.

1.11 Execution Role

A Lambda function often needs permission to access other AWS services.

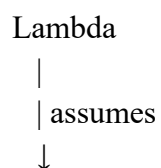
For example, suppose Lambda needs to read an S3 object:



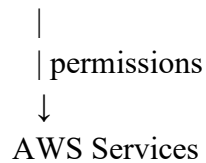
Lambda needs permission to perform that operation.

AWS uses an IAM execution role for this.

The relationship is:



IAM Execution Role



For example, the role could allow:

s3:GetObject

This means the Lambda function can retrieve objects from S3 according to the policy and resource scope.

Important interview point

The Lambda execution role is not the same thing as the user's IAM role.

The execution role is assumed by the Lambda service when the function executes.

1.12 Environment Variables

Environment variables allow us to store configuration values separately from our application code.

For example:

```
DATABASE_NAME = devdb
ENVIRONMENT    = dev
LOG_LEVEL      = INFO
```

Our code can retrieve them.

In Python:

```
import os

environment = os.environ.get("ENVIRONMENT")

print(environment)
```

This allows the same Lambda code to be deployed to different environments with different configuration values.

For example:

Development

`ENVIRONMENT = dev`

Testing

`ENVIRONMENT = test`

Production

`ENVIRONMENT = prod`

We don't have to modify the application code for every environment.

Important: Sensitive secrets should generally be protected using appropriate secret-management/encryption mechanisms rather than treating ordinary environment variables as a secure secret store.

1.13 Memory

Lambda allows us to configure the amount of memory allocated to a function.

For example:

128 MB

256 MB

512 MB

1024 MB

...

Memory allocation can affect the amount of CPU resources available to the function.

Therefore:

More memory



Potentially more CPU



Potentially faster execution

But it can also affect cost.

So memory should be selected according to application requirements and tested rather than simply choosing the largest value.

1.14 Timeout

Timeout defines the maximum amount of time Lambda is allowed to run before AWS terminates the invocation.

For example:

Timeout = 10 seconds

If the function does not finish within the configured timeout, the invocation can be terminated.

Example:

Lambda starts

```
|  
|---- 1 sec  
|---- 2 sec  
|---- 3 sec  
|  
X
```

Timeout reached

Therefore, timeout should be configured according to the expected workload.

1.15 Ephemeral Storage

Lambda provides temporary storage for the execution environment.

This is called ephemeral storage.

It can be used for temporary files during execution.

For example:

Lambda

```
|  
+---- /tmp  
|  
+---- temporary-file
```

The important word is temporary.

It should not be treated as permanent application storage.

If persistent storage is required, services such as S3 or databases should normally be considered.

1.16 Stateless Nature of Lambda

Lambda functions are generally designed to be stateless.

This means we should not depend on one invocation remembering application state from a previous invocation.

For example:

```
Invocation 1  
↓  
Lambda  
↓  
Invocation finishes
```

```
Invocation 2  
↓  
Lambda
```

We should not assume that data stored only in the execution environment will reliably be available for the next invocation.

For persistent information, use services such as:

S3

DynamoDB

RDS

ElastiCache

depending on the application requirement.

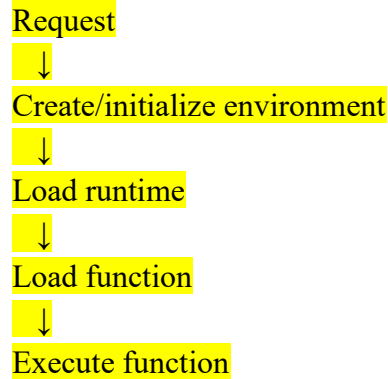
1.17 Cold Start vs Warm Start

This is an important Lambda interview topic.

Cold Start

When Lambda needs to initialize a new execution environment, there can be additional startup latency.

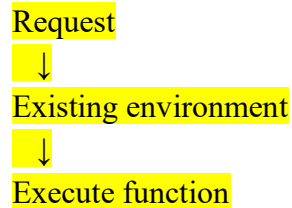
Conceptually:



This is called a cold start.

Warm Start

If an existing execution environment is available, Lambda may reuse it.



This can avoid some initialization work.

However, you should not design your application assuming that a particular execution environment will always remain warm.

1.18 Lambda Versions

Lambda supports versions of a function.

Suppose we initially deploy:

Version 1

Later we modify the code:

Version 2

We can maintain published versions so that specific deployments can be referenced.

Conceptually:

Lambda Function

```
|  
+---- Version 1  
|  
+---- Version 2  
|  
+---- Version 3
```

This is useful for deployment and release management.

1.19 Lambda Aliases

An alias is a named pointer to a particular published Lambda version.

For example:

Production

↓

Version 3

Testing

↓

Version 4

Instead of applications directly depending on version numbers, an alias can provide a stable name such as:

prod
test
dev

This is useful in deployment strategies.

1.20 Concurrency

Concurrency refers to the number of Lambda invocations being processed at the same time.

Suppose 100 users send requests at approximately the same time:

100 requests

|

v

Lambda

|

+--- Invocation 1

+--- Invocation 2

+--- Invocation 3

+--- ...

+--- Invocation 100

Lambda can process multiple invocations concurrently, subject to applicable account, function, and service limits.

Concurrency is important for understanding scalability and controlling how many executions can occur simultaneously.

1.21 Lambda Pricing Concept

Lambda follows a usage-based pricing model.

You generally pay based on factors such as:

- Number of requests
- Amount of compute time
- Memory configuration

The exact pricing depends on the AWS Region, architecture, usage, and applicable AWS pricing/free-tier terms.

The major concept to remember is:

Lambda is usage-based rather than requiring you to pay for a continuously running EC2 server simply to keep the application available.

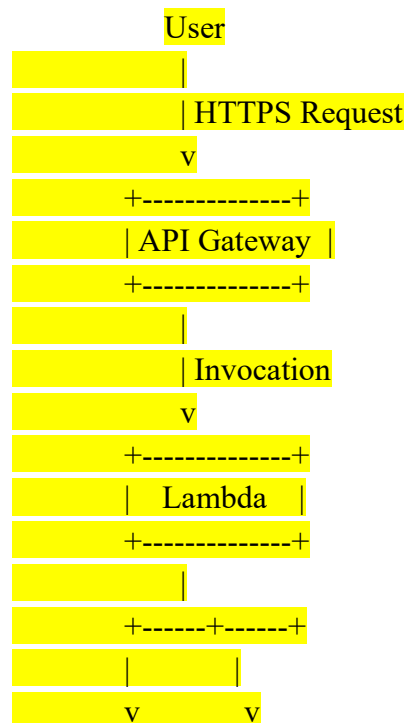
For your hands-on lab, we should still monitor resources and clean up anything that is no longer required.

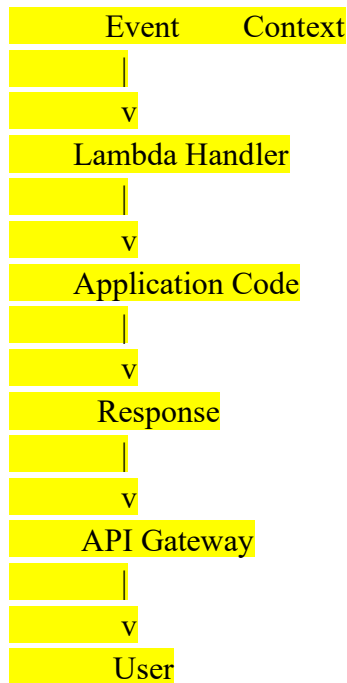
1.22 Complete Lambda Flow

Now combine everything.

Suppose we create an API using API Gateway.

The complete flow becomes:





This is the architecture we will actually build during the hands-on activity.

1.23 Real-Time Example

Imagine we are building a student application.

A student opens:

<https://api.example.com/student>

The request goes to:

API Gateway

API Gateway invokes:

Lambda

Lambda executes:

```
def lambda_handler(event, context):  
    return {  
        "statusCode": 200,
```

```
"body": "Student details retrieved successfully"
}
```

The response travels back:

Lambda

↓

API Gateway

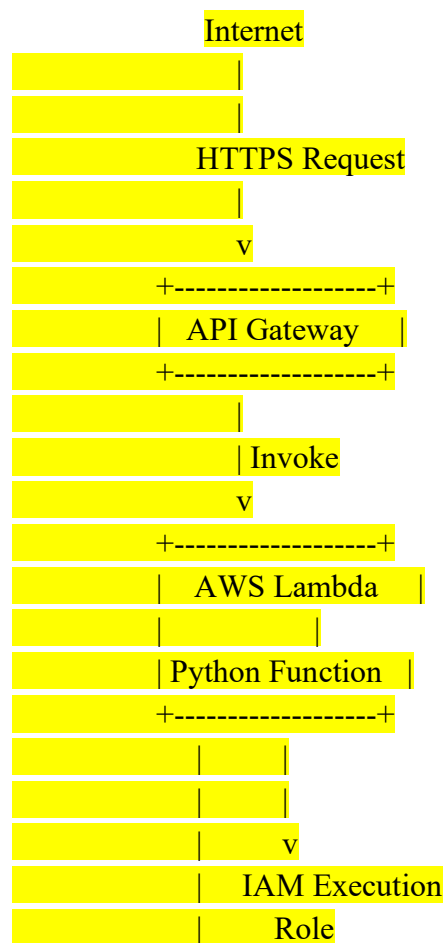
↓

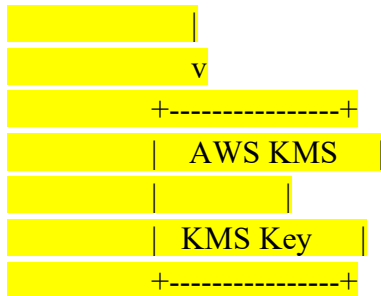
Student

There is no EC2 server that we had to manually create and maintain for this backend function.

1.24 Lambda Architecture for Our Assignment

Our final learning architecture will be:





Here:

API Gateway

Receives the HTTP request.

Lambda

Runs our application logic.

IAM

Provides Lambda with permissions required to access AWS resources.

KMS

Provides cryptographic key management and can be used for encryption/decryption operations.

1.25 Lambda — Important Terms to Remember

For your mentor/interview, remember this table:

Term	Meaning
Lambda	Serverless compute service
Function	Unit of executable Lambda code
Runtime	Environment used to execute the code
Handler	Entry point of the Lambda code
Event	Input sent to Lambda
Trigger	Mechanism/service that invokes Lambda
Context	Information about the current invocation
Execution Role	IAM role Lambda assumes to access AWS resources

Environment Variables	Configuration values available to the function
Memory	Memory allocated to the function
Timeout	Maximum execution duration
Ephemeral Storage	Temporary storage available during execution
Version	Published version of Lambda code/configuration
Alias	Named pointer to a published version
Concurrency	Number of simultaneous Lambda executions
Cold Start	Initialization of a new execution environment
Warm Start	Reuse of an existing execution environment

"What is AWS Lambda?"

AWS Lambda is a serverless, event-driven compute service that allows us to run application code without provisioning or managing servers. We create a Lambda function, select a runtime such as Python, provide the application code and configure a handler. Lambda is invoked by events or triggers such as API Gateway, S3, EventBridge, SQS or SNS. When the function executes, AWS provides the event and context objects to the handler. Lambda can access other AWS services using an IAM execution role. We can configure memory, timeout, environment variables and ephemeral storage, and Lambda supports versions, aliases and concurrent executions.

Task 1 — Create an AWS Lambda Function

Objective

To create an AWS Lambda function using the Python runtime and understand the basic configuration required to execute serverless code.

Steps Performed:

1. Opened the AWS Management Console.
2. Navigated to the **AWS Lambda** service.
3. Selected **Create function**.
4. Selected **Author from scratch**.
5. Entered the function name as `devops-lambda-hello`.
6. Selected **Python 3.14** as the runtime.
7. Used the default Lambda execution role with basic Lambda permissions.
8. Kept the default x86_64 architecture.
9. Did not configure VPC, Function URL, KMS, code signing, or other advanced settings because they are not required for the initial Lambda function.
10. Created the Lambda function.
11. Verified the default Python source code and Lambda handler.

The screenshot shows the AWS Lambda 'Create function' page in the AWS Management Console. The breadcrumb navigation at the top reads 'Lambda > Functions > Create function'. The main heading is 'Create function' with an 'Info' link. Below this, a subtext says 'Choose one of the following options to create your function.' There are three radio button options: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. The 'Author from scratch' option has a subtext: 'Start with a simple Hello World example.' Below these options is the 'Basic information' section. It contains a 'Function name' field with the value 'devops-lambda-hello' and a note: 'Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).' The 'Runtime' section shows 'Python 3.14' selected from a dropdown menu. The 'Permissions' section states: 'By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose Custom execution role under Additional settings.' At the bottom is the 'Custom settings' section. On the right side of the console, there is a sidebar with 'Info' and 'Tutorials' tabs. The 'Tutorials' tab is active, showing a 'Choose a tutorial' section with the heading 'Create a simple web app' and a list of steps: 'Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage' and 'Invoke your function through its function URL'. There is a 'Learn more' link and a 'Start tutorial' button.

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

Lambda > Functions > Create function

0

Custom settings

Info

Durable execution

Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider

Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

Additional settings

Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

General

Info

ARM64 architecture

Use ARM64 instead of x86_64 (default).

Custom execution role

IAM permissions

devops-lambda-hello-role-a3nfpoww

Edit

Networking

Info

Function URL

Assign HTTPS endpoints.

Tenant isolation mode

Guarantee separate execution environments for each tenant invoking the function.

VPC

Connect to a VPC to securely access private resources.

Security & governance

Info

Info

Tutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

^

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

Lambda > Functions > devops-lambda-hello

0

devops-lambda-hello

Throttle

Copy ARN

Actions

Export to Infrastructure Composer

Download

Function overview

Info

Diagram

Template

devops-lambda-hello

Layers

(0)

+ Add trigger

+ Add destination

Function ARN

arn:aws:lambda:us-east-1:331116047691:function:devops-lambda-hello

Description

-

Last modified

5 minutes ago

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Info

Open in Visual Studio Code

Update

EXPLORER

DEVOPS-LAMBDA-HELLO

lambda_function.py

lambda_function.py

1 import json

2

3 def lambda_handler(event, context):

Info

Tutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

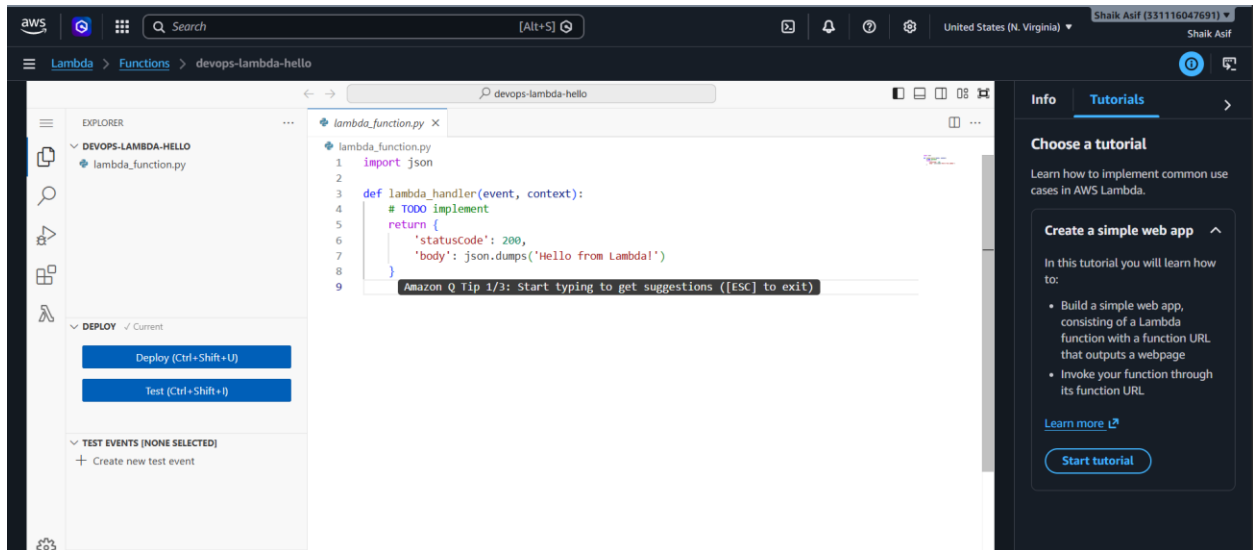
^

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial



Observation

The Lambda function was successfully created. AWS automatically provided the Python runtime environment and configured the function with a Lambda execution role. The function contains a `lambda_handler` entry point that accepts an event and context and returns a response.

Result

The devops-lambda-hello AWS Lambda function was successfully created and is ready for invocation and testing.

Task 2: Invoke and Test Lambda Function

Objective

To create a test event, invoke the Lambda function, verify its execution result, and understand how Lambda receives event data and produces a response.

Steps Performed

1. Opened the previously created devops-lambda-hello Lambda function.
2. Selected **Create new test event**.
3. Created a test event named lambda-test-event.

4. Provided the following JSON input:

```
{
  "name": "Asif",
  "purpose": "Lambda testing"
}
```

5. Saved the test event.

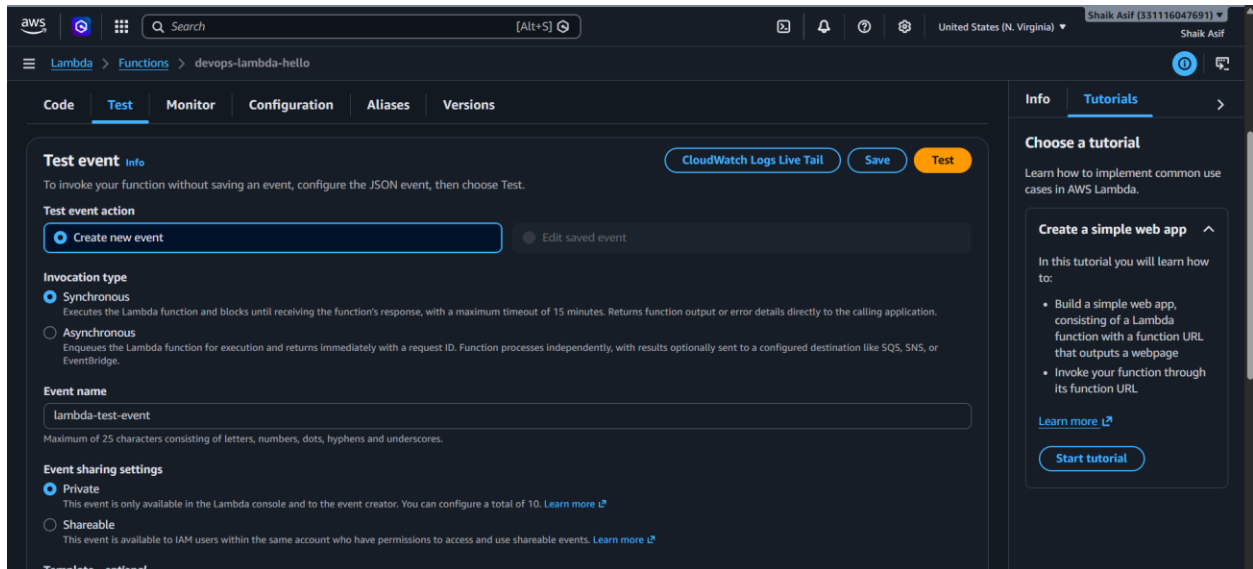
6. Invoked the Lambda function using the **Test** option.

7. Verified that the Lambda function executed successfully.

8. Observed the returned response containing HTTP status code 200.

9. Reviewed the execution information and Lambda logs.

10. Verified that Lambda received the test JSON as the event parameter.



aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

Lambda > Functions > devops-lambda-hello

0

Event sharing settings

Private

This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

Shareable

This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

Generate with Amazon Q

Event JSON

Format JSON

1 {

2 "name": "Asif",

3 "purpose": "Lambda testing"

4 }

Info

Tutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

CloudWatch > Log management > /aws/lambda/devops-lambda-hello

0

CloudWatch

Favorites and recents

Ingestion

Dashboards

Alarms

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Logs

Log Management

Log Analytics

Log Anomalies

Metrics

Query Studio

/aws/lambda/devops-lambda-hello

Actions

View in Logs Insights

Start tailing

Search log group

Log group details

Log class

Standard

ARN

arn:aws:logs:us-east-1:331116047691:log-group:/aws/lambda/devops-lambda-hello:

Creation time

3 minutes ago

Retention

Never expire

Stored bytes

-

Metric filters

0

Subscription filters

0

Contributor Insights rules

-

KMS key ID

-

Deletion protection

Off

Data protection

-

Sensitive data count

-

Custom field indexes

Configure

Transformer

Configure

Anomaly detection

Configure

Bearer token authentication

Off

Syslog ingestion

Info

Configure

Log streams

Tags

Data protection

Anomaly detection

Metric filters

Subscription filters

Contributor Insights

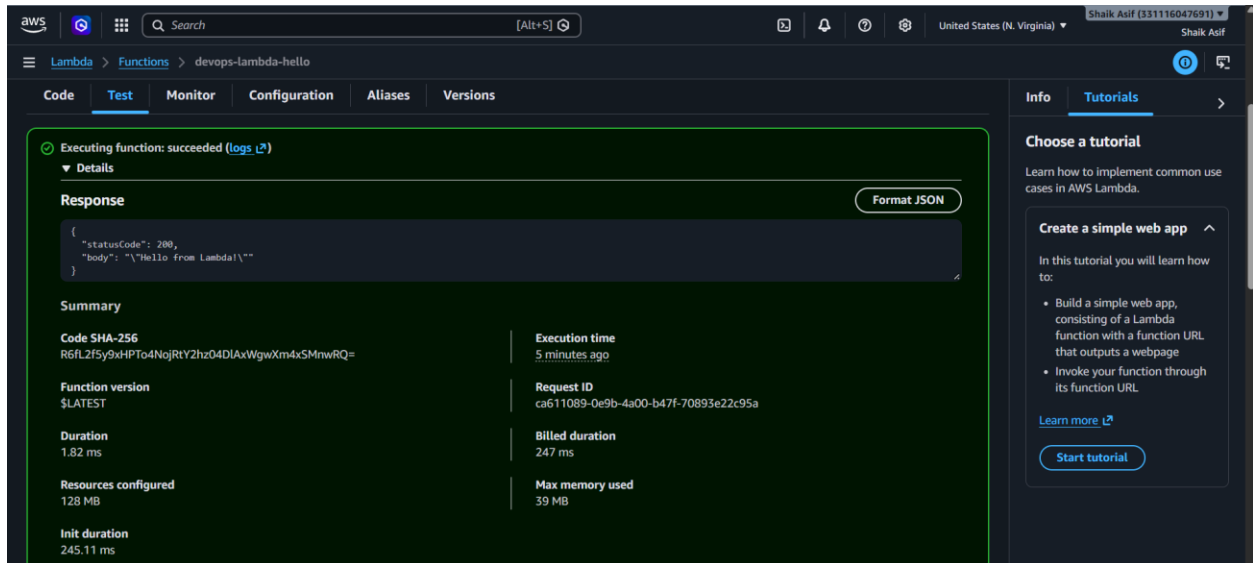
Field in

Log streams (1)

Delete

Create log stream

Search all log streams



Observation

The Lambda function successfully executed when invoked with the configured test event. The JSON test data was passed to the Lambda handler through the event parameter. The function executed the Python code and returned a successful response. Execution information and logs were also generated for the invocation.

Result

The devops-lambda-hello Lambda function was successfully invoked and tested. The execution completed successfully and returned the expected response.

Task 3: Configure Lambda Function

Objective

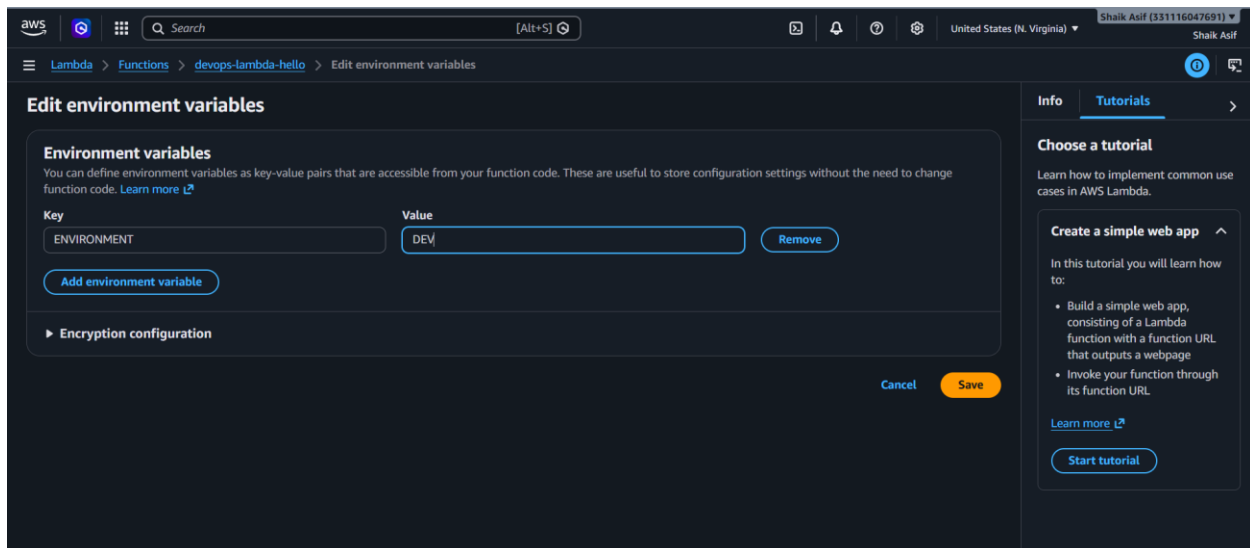
To understand and configure the important AWS Lambda settings including runtime, handler, memory, timeout, ephemeral storage, environment variables, and IAM execution role.

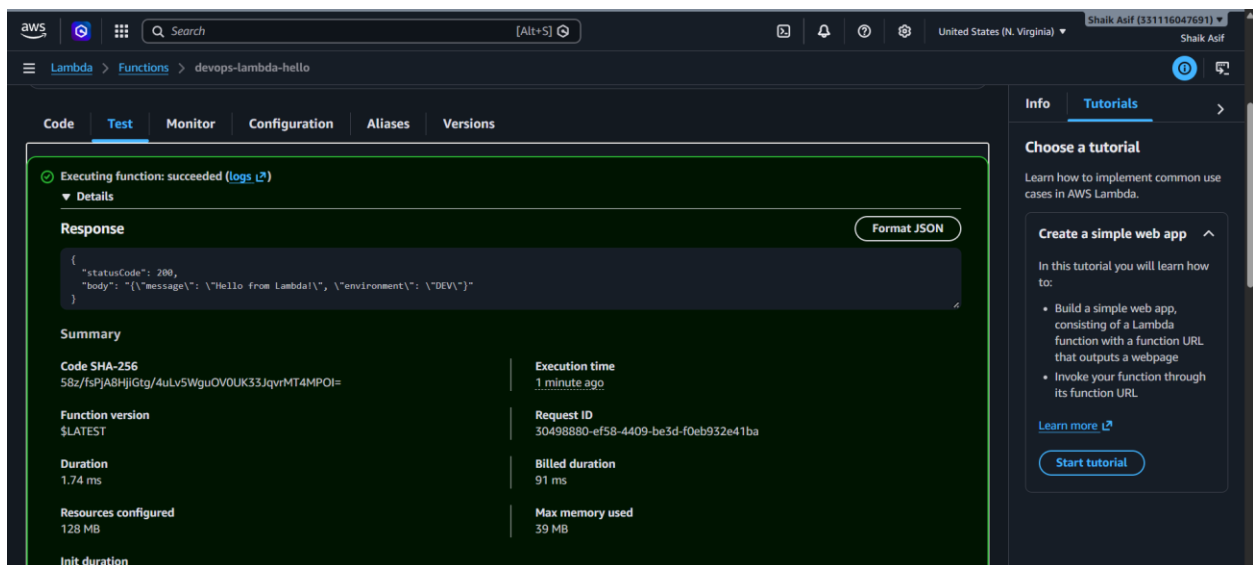
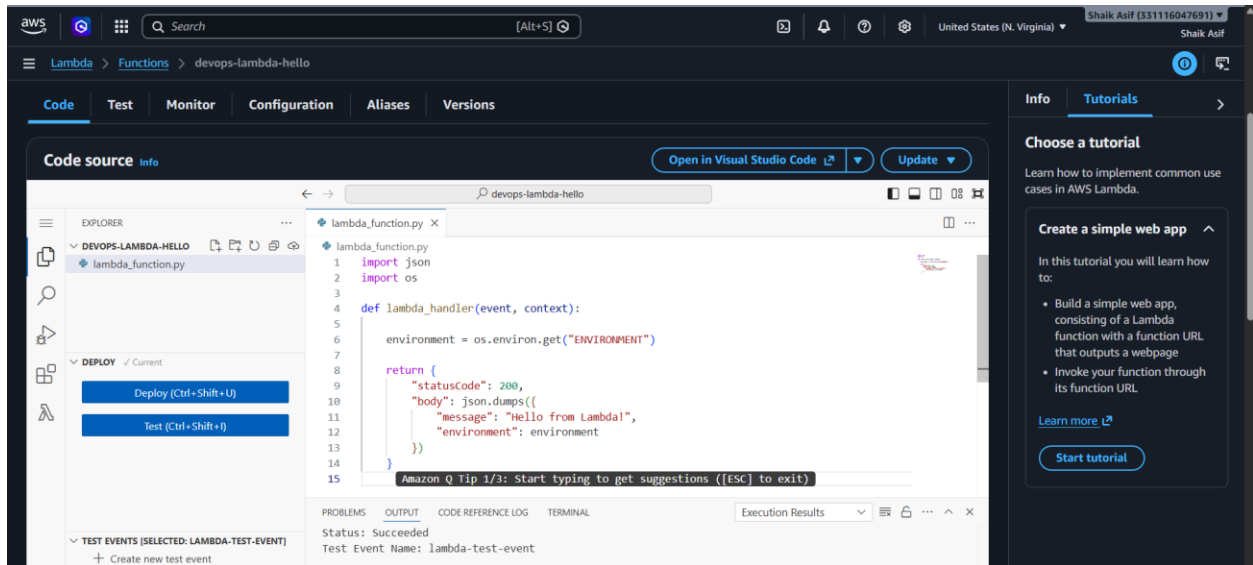
Steps Performed

1. Opened the `devops-lambda-hello` Lambda function.
2. Navigated to **Configuration**.
3. Reviewed the **General configuration** settings.
4. Reviewed the configured Python runtime and Lambda handler.
5. Reviewed the IAM execution role associated with the function.
6. Added the environment variable:

`ENVIRONMENT = DEV`

7. Updated the Lambda code to retrieve the environment variable using Python's `os.environ`.
8. Deployed the updated code.
9. Invoked the Lambda function using the existing test event.
10. Verified the returned response.





Observation

Lambda provides configurable execution parameters such as memory, timeout, ephemeral storage, runtime, and handler. Environment variables can be used to provide application configuration without hard-coding values into the source code. The Lambda execution role controls the AWS resources that the function is permitted to access.

Result

The Lambda function was successfully configured with an environment variable and tested. The function successfully retrieved the configured ENVIRONMENT value during execution and returned it in the response.

Task 4: Integrate AWS Lambda with API Gateway

Objective

To create an Amazon API Gateway HTTP API and integrate it with the existing devops-lambda-hello Lambda function so that a client can invoke Lambda using an HTTP request.

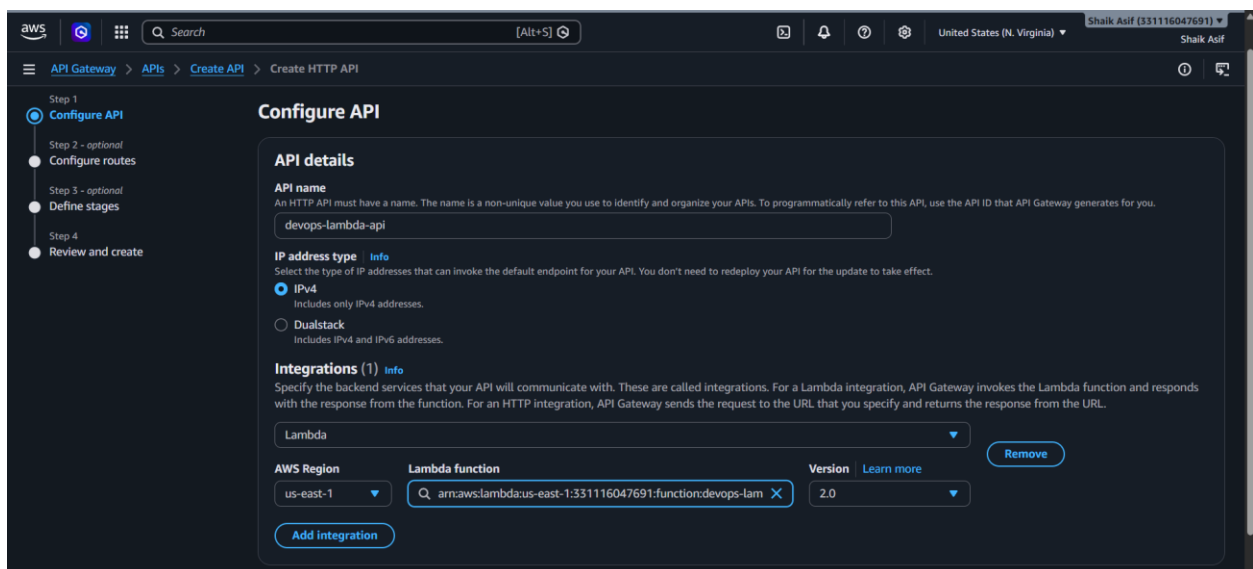
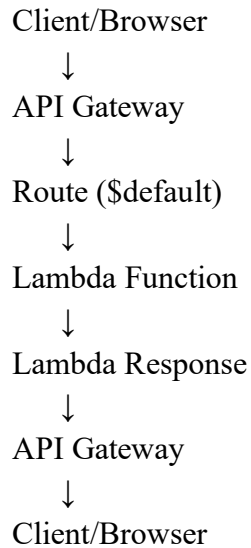
Steps Used

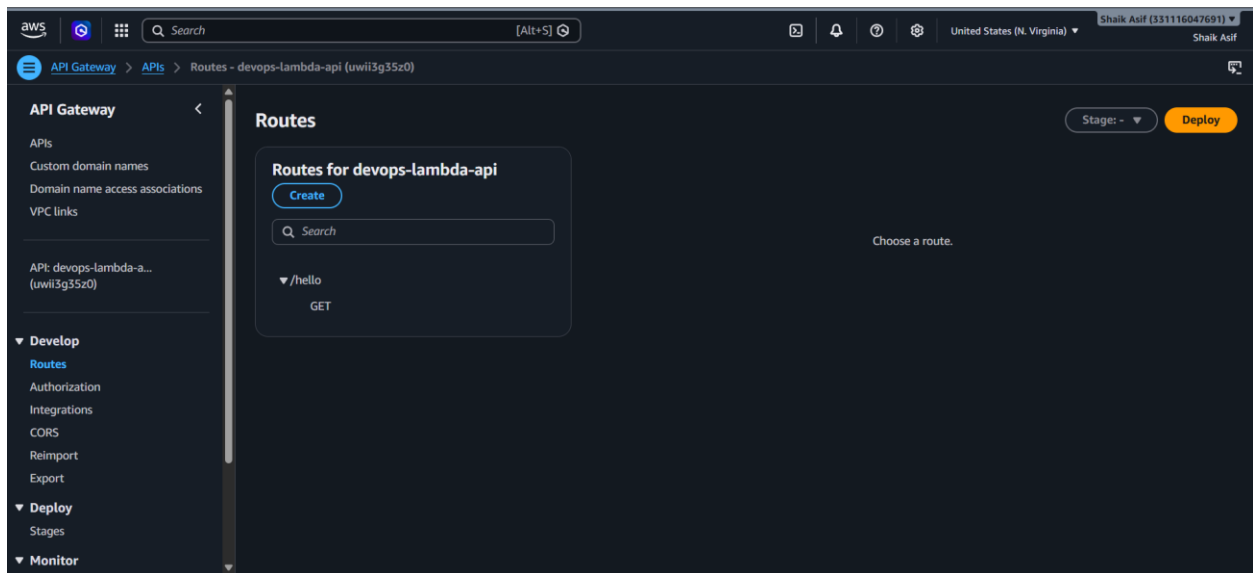
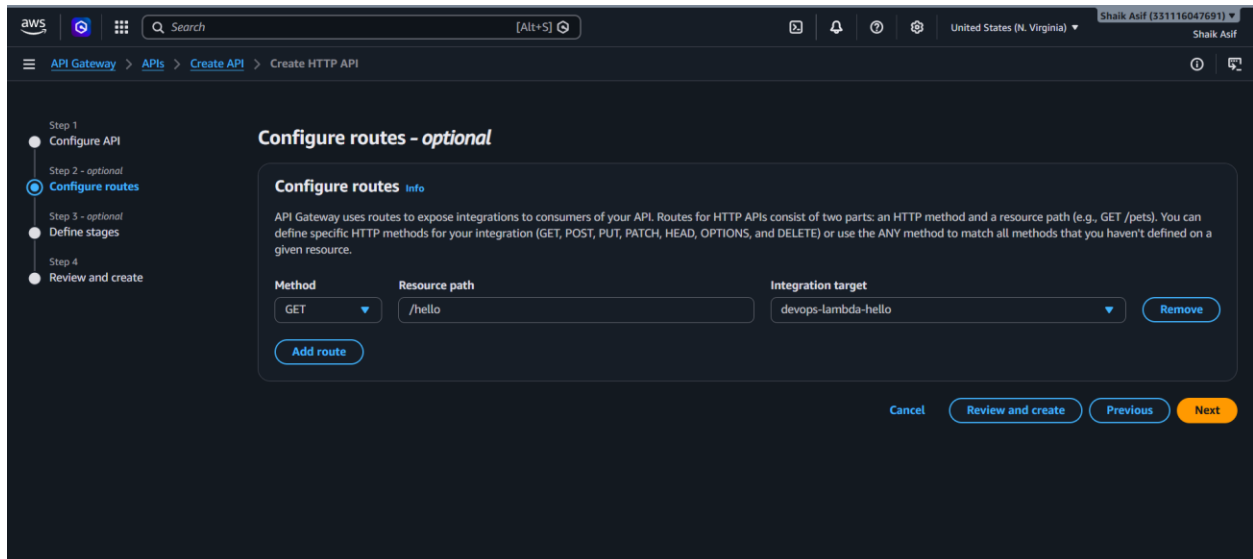
1. Opened **AWS Management Console** → **API Gateway**.
2. Clicked **Create API** and selected **HTTP API**.
3. Created the API with the name:devops-lambda-api
4. Selected the existing Lambda function:devops-lambda-helloas the backend integration.
5. Created the initial route:GET /hello
6. Created a stage named:dev
7. Obtained the API Gateway Invoke URL:https://uwii3g35z0.execute-api.us-east-1.amazonaws.com/dev
8. Initially tested the base URL and received:{"message":"Not Found"}because the configured route was /hello, while the request was being sent to the stage root.
9. Created a **\$default** route so that requests not matching another specific route could be handled.
10. Attached the existing Lambda function:devops-lambda-helloto the \$default route.
11. Observed that **Automatic Deployment was disabled**, so the route change was not immediately available.
12. Manually deployed the updated API to the **dev** stage.
13. Opened the Invoke URL again:https://uwii3g35z0.execute-api.us-east-1.amazonaws.com/dev
14. Verified that the request successfully reached the Lambda function and returned the Lambda response.

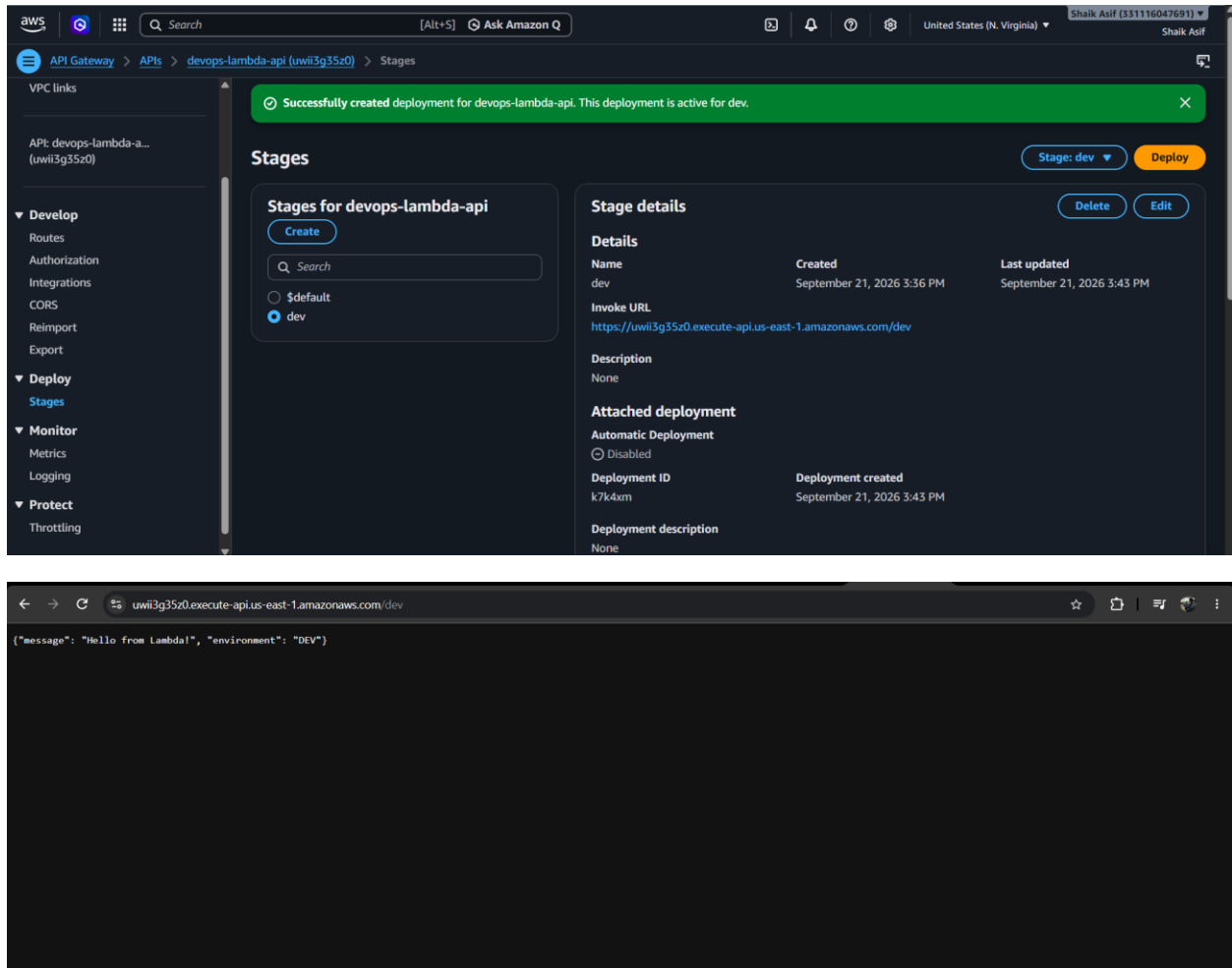
Observation

- API Gateway provides an HTTP endpoint through which clients can invoke the Lambda function.
- The API contains routes that determine how incoming HTTP requests are handled.
- The GET /hello route provides a specific /hello endpoint.

- The \$default route allows the base stage URL to be handled when no more specific route matches.
- The \$default route was connected to devops-lambda-hello.
- Since **automatic deployment was disabled**, changes to the API configuration had to be manually deployed to the dev stage.
- After deployment, the API Gateway URL successfully invoked the Lambda function.
- This demonstrated the complete request flow:







Result / Conclusion

Successfully integrated the AWS Lambda function devops-lambda-hello with Amazon API Gateway. Created the dev stage, configured API Gateway routes and Lambda integration, deployed the changes, and successfully invoked the Lambda function through the API Gateway Invoke URL.

Amazon API Gateway

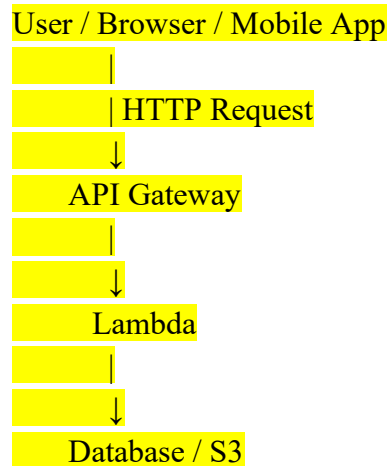
1. What is Amazon API Gateway?

Amazon API Gateway is an AWS service used to create, publish, secure, monitor, and manage APIs.

In simple words:

API Gateway is the front door through which applications can communicate with backend services such as Lambda.

Think about a real application:



For example, when a user opens:

<https://example.com/users>

the request can reach API Gateway, which determines:

"This is a GET request for /users. Which backend should handle it?"

It can then send the request to Lambda.

2. Why do we need API Gateway?

Already created a Lambda function:

devops-lambda-hello

Lambda can execute code, but Lambda itself isn't primarily an HTTP API management service.

API Gateway gives us things such as:

- HTTP/HTTPS endpoints
- Routes
- Authentication and authorization
- Request handling

- Throttling
- CORS
- Monitoring
- Integration with Lambda and other AWS services

So:

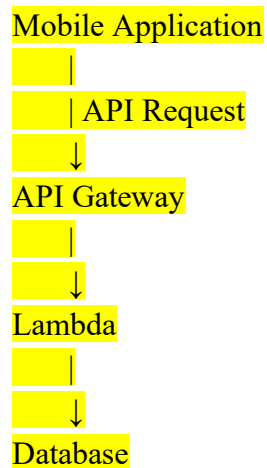
Lambda = Runs the backend code

API Gateway = Provides/manages the API interface

3. What is an API?

An **API** is an interface that allows one application to communicate with another application or service.

For example:



The mobile application doesn't need to know how the backend code works.

It simply sends a request such as:

GET /users

and receives a response.

4. What is a Route?

A **route** tells API Gateway how to handle a particular request.

A route consists of:

HTTP Method + Path

Examples:

GET /users

POST /users

GET /users/{id}

DELETE /users/{id}

In our previous implementation, we created:

GET /hello

and also:

\$default

Meaning of GET

GET normally means:

"I want to retrieve/read something."

For example:

GET /users

means retrieve users.

POST

Normally used to create/send data:

POST /users

PUT/PATCH

Used to update data.

DELETE

Used to delete data.

5. What is an Integration?

This is another important concept.

The **integration** tells API Gateway which backend should process the request.

For example:

GET /hello

|

↓

Lambda Integration

|

↓

devops-lambda-hello

So:

Route = What request came in?

Integration = Where should that request go?

6. What is an Invoke URL?

When we create an API Gateway API and stage, AWS provides an **Invoke URL**.

Our previous URL was:

<https://uwii3g35z0.execute-api.us-east-1.amazonaws.com/dev>

A client can send a request to that URL.

With a route such as:

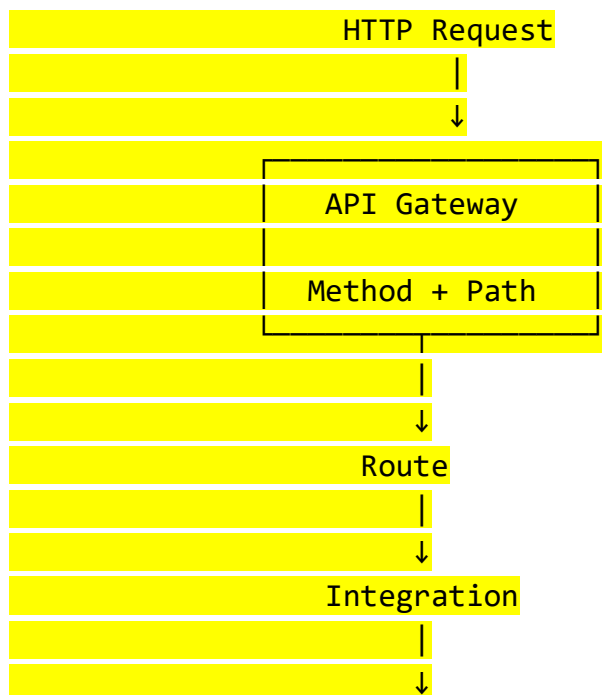
GET /hello

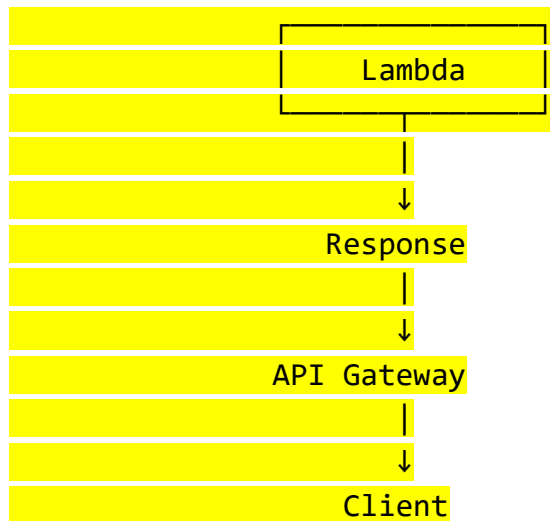
the request becomes:

<https://uwii3g35z0.execute-api.us-east-1.amazonaws.com/dev/hello>

The API Gateway receives the request and determines which route matches it.

7. API Gateway Request Flow



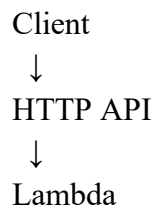


8. API Gateway Types

There are two major API Gateway API types you should know:

HTTP API

Designed for simpler, lower-cost HTTP APIs.



This is what we used in our Lambda exercise.

REST API

Provides more advanced API-management capabilities and configuration options.



For interviews, remember:

HTTP API is generally simpler and designed for HTTP API workloads, while REST API provides more extensive API-management capabilities.

9. Important API Gateway Concepts

For your documentation and interviews, learn these:

API

The API Gateway resource containing your API configuration.

Route

HTTP method + path.

Example:

GET /users

Integration

Backend connected to the route.

Example:

Lambda

Stage

A named deployment environment/version of the API.

Examples:

dev

test

prod

Invoke URL

The URL clients use to call the API.

Authorization

Controls who can access the API.

Authentication

Verifies who the caller is.

CORS

Controls whether browser-based applications from different origins can access the API.

Throttling

Controls the number/rate of requests allowed.

Deployment

Makes API configuration available through a stage.

The screenshot displays the AWS API Gateway console interface for creating a new HTTP API. The breadcrumb navigation shows the path: API Gateway > APIs > Create API > Create HTTP API. A sidebar on the left indicates the current step is 'Step 1: Configure API', with subsequent steps being 'Step 2 - optional: Configure routes', 'Step 3 - optional: Define stages', and 'Step 4: Review and create'.

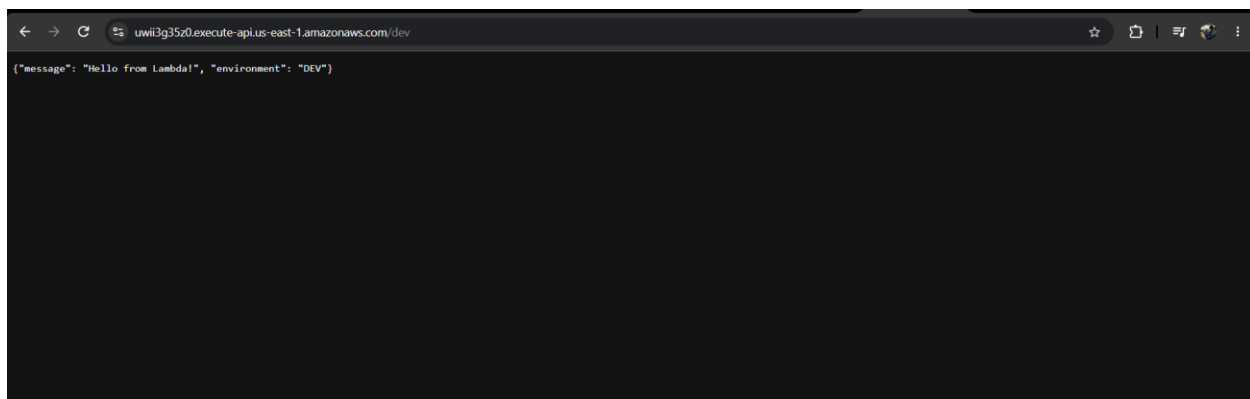
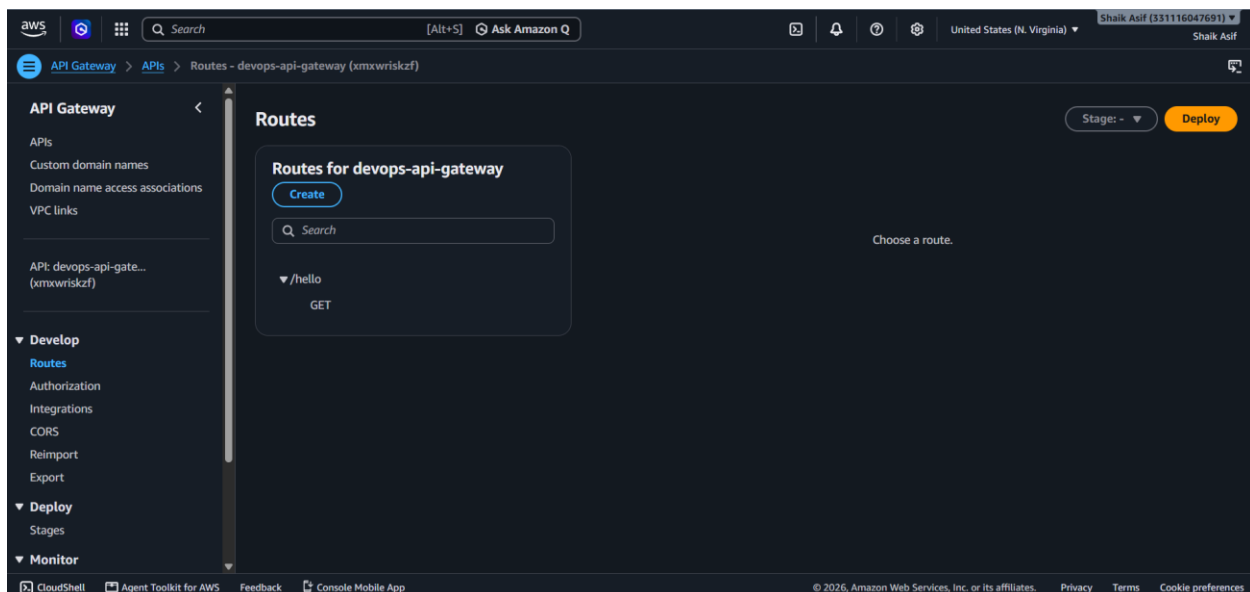
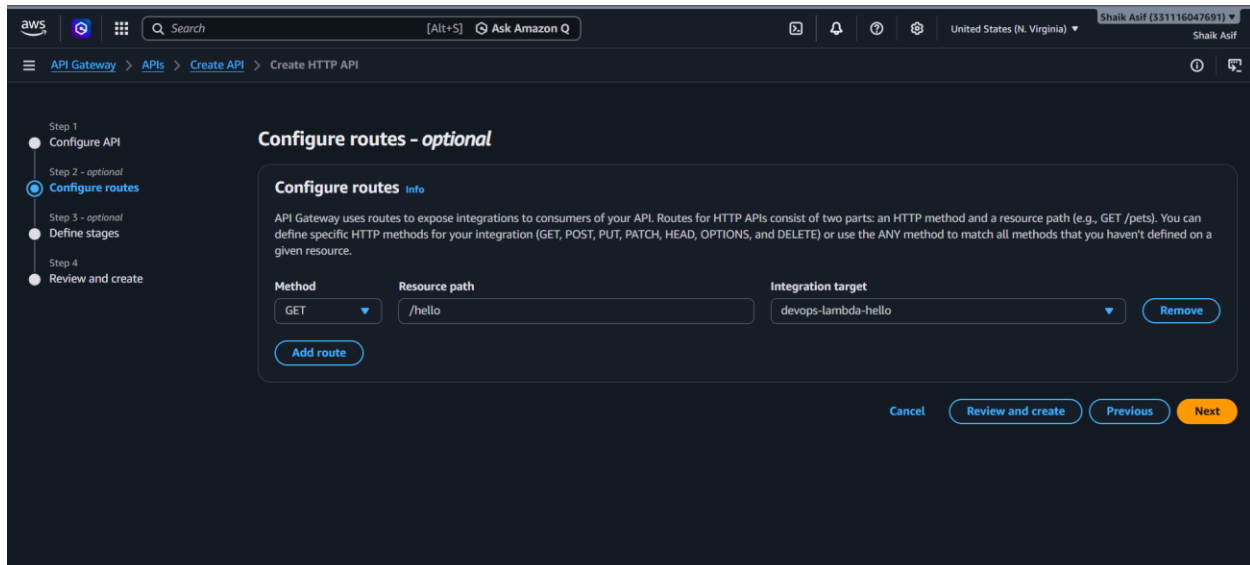
The main content area is titled 'Configure API' and contains the following sections:

- API details**
 - API name:** A text input field containing 'devops-api-gateway'. A note states: 'An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.'
 - IP address type:** A section with a link to 'Info'. It includes two radio button options:
 - IPv4:** Selected. Includes only IPv4 addresses.
 - Dualstack:** Includes IPv4 and IPv6 addresses.
- Integrations (1)** with an 'Info' link. A note explains: 'Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.'

Below the integrations section, there is a table-like structure for the existing integration:

AWS Region	Lambda function	Version	Action
us-east-1	arn:aws:lambda:us-east-1:331116047691:function:devops-lam	2.0	Remove

At the bottom of the integration section, there is an [Add integration](#) button.



Result / Conclusion

Successfully integrated the AWS Lambda function devops-lambda-hello with Amazon API Gateway. Created the dev stage, configured API Gateway routes and Lambda integration, deployed the changes, and successfully invoked the Lambda function through the API Gateway Invoke URL.

AWS Key Management Service (AWS KMS)

1. What is AWS KMS?

AWS Key Management Service (AWS KMS) is a managed AWS service used to **create, store, control, and manage cryptographic keys** that are used to protect data.

In simple words:

AWS KMS is a service that manages encryption keys and controls who can use those keys to encrypt and decrypt data.

Instead of creating and maintaining our own encryption-key management system, AWS provides KMS to securely manage the keys.

Simple example

Suppose we have sensitive data:

customer-data.txt

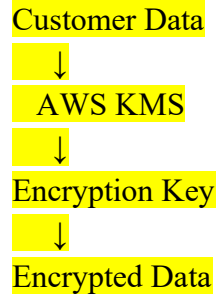
Without encryption:

customer-data.txt



Readable information

With encryption:



Only an authorized user/application/service that has permission to use the KMS key can decrypt the protected data.

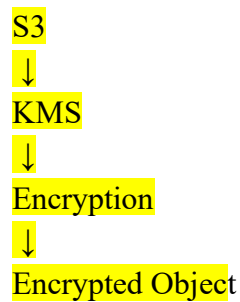
2. Why do we use AWS KMS?

Organizations use KMS to protect sensitive information and control access to encryption keys.

Common use cases include:

- Encrypting S3 objects
- Encrypting EBS volumes
- Encrypting RDS databases
- Encrypting application data
- Encrypting secrets and configuration
- Protecting backups
- Controlling access to encryption keys
- Auditing key usage

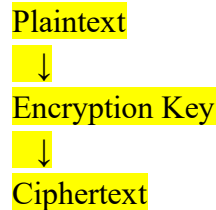
For example:



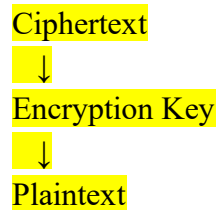
3. What is an encryption key?

An **encryption key** is cryptographic information used by an encryption algorithm to transform readable data into protected data.

For example:



To retrieve the original data:

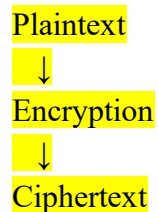


KMS manages these keys rather than requiring us to manage the underlying key material ourselves in typical AWS KMS usage.

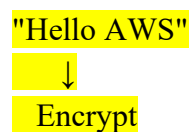
4. Encryption and Decryption

Encryption

Encryption converts readable information (**plaintext**) into protected information (**ciphertext**).



Example:



↓
"8fA92...."

Decryption

Decryption converts ciphertext back into the original plaintext.

Ciphertext
↓
Decryption
↓
Plaintext

KMS provides APIs such as:

Encrypt

Decrypt

for supported cryptographic operations.

5. Important AWS KMS Components

These are the concepts you should know for interviews and your documentation.

A. KMS Key

A **KMS key** is the primary resource used by KMS for cryptographic operations.

It can be used to:

- Encrypt
- Decrypt
- Generate data keys
- Protect other encryption keys

B. Customer Managed Key

A **customer managed key** is a KMS key that you create and manage in your AWS account.

You can control things such as:

- Key policy
- IAM permissions
- Alias
- Rotation settings
- Key lifecycle

Example:

alias/devops-kms-key

For our hands-on lab, we'll create a **customer managed symmetric KMS key**.

C. AWS Managed Key

An **AWS managed key** is created and managed by AWS for a specific AWS service.

For example, an AWS service can use an AWS managed KMS key to encrypt resources on your behalf.

The important distinction is:

AWS managed key

→ AWS manages it

Customer managed key

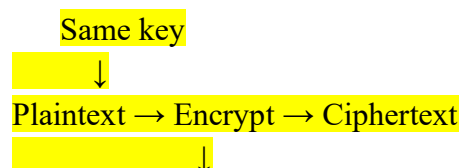
→ Customer has more control over it

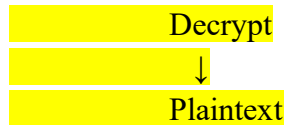
6. Symmetric vs Asymmetric KMS Keys

This is an important interview topic.

Symmetric key

A symmetric key uses the **same key material for encryption and decryption**.





Example:

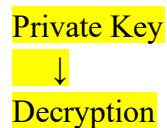
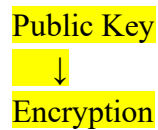
Symmetric KMS key

Key usage: Encrypt/Decrypt

This is commonly used for data encryption.

Asymmetric key

An asymmetric key uses a **public key and private key pair**.



Asymmetric KMS keys can also be used for digital signing and verification, depending on the key configuration.

For our lab

We'll use:

Symmetric → Encrypt and decrypt

because it is easier to understand and is widely used for encrypting AWS resources.

7. KMS Key Alias

A KMS key has a unique key ID, which can be difficult to remember.

For example:

12345678-abcd-1234-abcd-123456789012

We can create a friendly alias:

alias/devops-kms-key

So:

Long Key ID



alias/devops-kms-key

Aliases make keys easier to identify and reference.

8. KMS Key Policy

This is **very important**.

A KMS key policy is a **resource-based policy attached to the KMS key**.

It defines which principals can perform actions on the key.

For example:

IAM User



KMS Key Policy



Allow Encrypt



KMS Key

Remember this sentence for interviews:

KMS key policy controls who can administer or use the KMS key.

KMS authorization can involve both **key policies and IAM policies**, depending on how the permissions are configured.

9. Key Administrator vs Key User

These are different responsibilities.

Key Administrator

A key administrator can manage the KMS key.

For example:

- Configure key policy
- Enable/disable key
- Configure rotation
- Schedule deletion

Key User

A key user can use the key for cryptographic operations.

For example:

Encrypt

Decrypt

GenerateDataKey

Simple way to remember:

Administrator = manages the key

User = uses the key

10. Key Rotation

Key rotation means periodically changing the cryptographic key material used by a KMS key according to AWS's key-management process.

Why?

It reduces the amount of data protected by a particular key material over time and supports security best practices.

For supported KMS keys, AWS provides **automatic key rotation**.

Important:

Key rotation does **not** mean that you need to manually replace the KMS key everywhere each time rotation occurs.

The logical KMS key remains available while AWS manages the rotation of its key material.

11. Key States

A KMS key can have different states during its lifecycle.

Common examples include:

Enabled

Disabled

PendingDeletion

PendingImport

The most common state during normal usage is:

Enabled

Only an enabled key can perform applicable cryptographic operations.

12. KMS + S3 Example

This is one of the most important real-world examples.

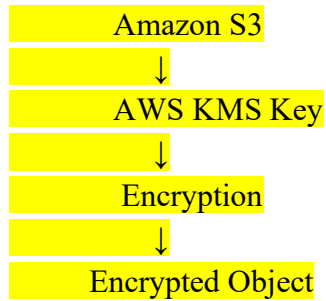
Suppose we upload:

customer-data.csv

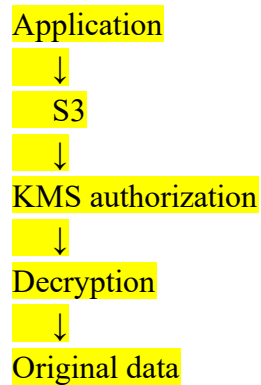
to S3.

Architecture:





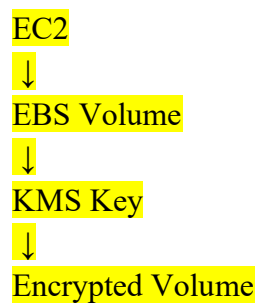
When an authorized application retrieves the object:



So S3 stores the object with encryption, while KMS manages the encryption key.

13. KMS + EBS Example

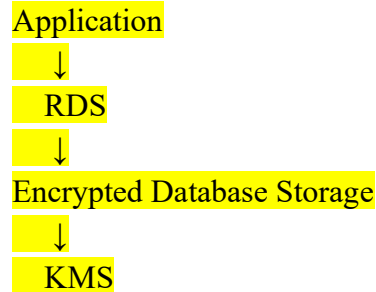
EBS volumes can also use KMS encryption.



This protects data stored on the EBS volume.

14. KMS + RDS Example

RDS databases can also use KMS encryption.



KMS helps protect the database storage and related encrypted resources.

15. KMS + CloudTrail

KMS API activity can be recorded through **AWS CloudTrail**.

For example:



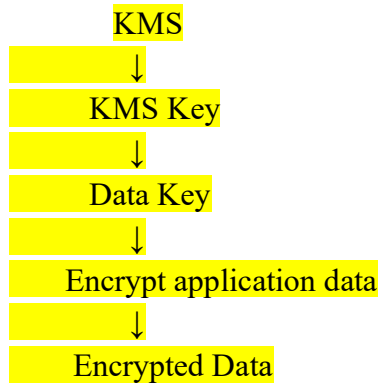
This helps organizations investigate and audit key-related activity.

16. Envelope Encryption

This is an important concept for a DevOps interview.

Instead of using a KMS key directly to encrypt large amounts of application data, applications commonly use **envelope encryption**.

Basic flow:



The data key encrypts the actual data, while KMS protects the data key.

Simple idea:

KMS protects the key; the data key protects the data.

This is useful because applications can efficiently encrypt larger amounts of data without sending all of that data through the KMS encryption API.

17. AWS KMS CLI

Since you're going to implement this through CLI, these are the important commands we'll practice:

`aws kms list-keys`

List KMS keys.

`aws kms describe-key`

View key information.

`aws kms list-aliases`

View aliases.

`aws kms create-key`

Create a customer managed KMS key.

aws kms create-alias

Create a friendly alias.

aws kms encrypt

Encrypt data.

aws kms decrypt

Decrypt data.

aws kms get-key-policy

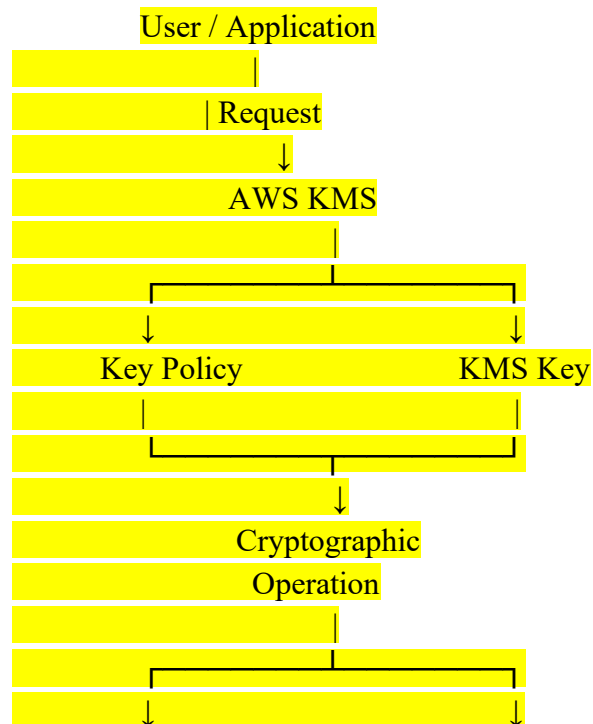
View the key policy.

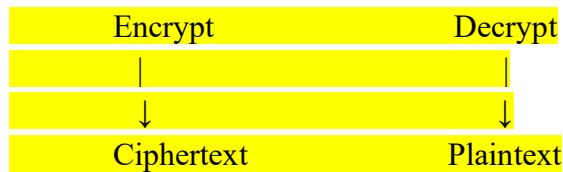
aws kms get-key-rotation-status

Check rotation status.

18. Overall KMS Architecture

For your documentation, you can use this flow:





Create and Configure a Customer-Managed AWS KMS Key and Integrate it with Amazon S3 Using SSE-KMS

KMS — Steps Used

1. Create Customer Managed KMS Key

- Opened **AWS KMS** in us-east-1.
- Navigated to **Customer managed keys** → **Create key**.
- Selected:
 - **Key type:** Symmetric
 - **Key usage:** Encrypt and decrypt
 - **Key spec:** SYMMETRIC_DEFAULT
 - **Regionality:** Single Region
- Created the KMS key.

2. Add Key Labels

- Created the alias:
alias/devops-kms-key
- Added the description:
KMS key for DevOps hands-on encryption lab
- No tags were added.

3. Configure Key Policy

- Reviewed the default KMS key policy.
- Initially no specific key administrator or key user was selected.
- The default policy contained **Enable IAM User Permissions** for the AWS account root.
- The policy was left unchanged.

4. Verify KMS Key

After creating the key, verified:

- Key status: **Enabled**
- Key type: **Symmetric**
- Key usage: **Encrypt and decrypt**
- Key manager: **Customer**
- Regionality: **Single Region**
- Alias: alias/devops-kms-key

5. Enable Automatic Key Rotation

- Opened the KMS key.
- Navigated to the key rotation settings.
- Enabled **Automatic key rotation**.
- CLI verification later confirmed:

KeyRotationEnabled: true

RotationPeriodInDays: 365

6. Create S3 Bucket

- Created an S3 bucket in us-east-1.
- Kept **Block Public Access** enabled.
- Configured default encryption as:

Server-side encryption with AWS KMS keys (SSE-KMS)

- Selected our customer-managed KMS key.

7. Configure S3 Bucket Key

- Enabled **S3 Bucket Key**.
- This helps reduce the number of KMS requests made by S3 and can reduce KMS request costs.

8. Verify S3 Encryption

The S3 bucket showed:

Encryption type:

Server-side encryption with AWS Key Management Service keys (SSE-KMS)

and displayed our KMS key ARN:

arn:aws:kms:us-east-1:331116047691:key/7827334e-c308-455d-aa3c-39c86643efa1

9. Add Key Administrator and Key User

- Added vpc-admin as a **Key Administrator**.
- Added vpc-admin as a **Key User**.

This helped demonstrate the difference between:

Key Administrator → manages the KMS key

Key User → uses the key for cryptographic operations

10. CLI Verification

Verified the KMS key using:

```
aws kms describe-key \  
--key-id 7827334e-c308-455d-aa3c-39c86643efa1 \  
--region us-east-1
```

The output confirmed:

```
Enabled: true  
KeyUsage: ENCRYPT_DECRYPT  
KeyState: Enabled  
KeyManager: CUSTOMER  
KeySpec: SYMMETRIC_DEFAULT  
MultiRegion: false
```

11. CLI Rotation Verification

Used:

```
aws kms get-key-rotation-status \
--key-id 7827334e-c308-455d-aa3c-39c86643efa1 \
--region us-east-1
```

Output confirmed:

KeyRotationEnabled: true
RotationPeriodInDays: 365

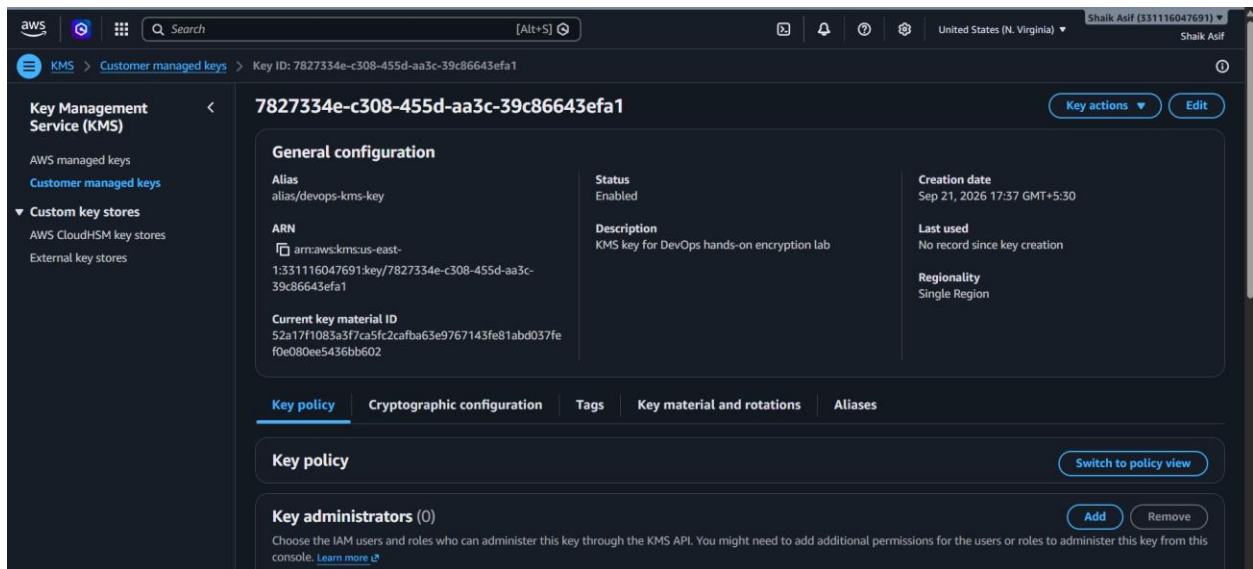
12. CLI Permission Observation

When running:

```
aws kms list-aliases --region us-east-1
```

AWS returned `AccessDeniedException` for `kms:ListAliases`.

This was a **permission issue with the IAM identity**, not a KMS key failure. The KMS key itself was successfully verified using `describe-key` and the `rotation-status` command.



Key Management Service (KMS)

AWS managed keys

Customer managed keys

Custom key stores

AWS CloudHSM key stores

External key stores

Key material and rotations

Manage AWS KMS key material and initiate automatic and on-demand key rotations.

Automatic key rotation

AWS KMS automatically rotates the key based on the rotation period that you define.

Status

Enabled

Rotation period

365

Date of last automatic rotation

-

Next rotation date

Sep 21, 2027

On-demand key rotation

Immediately initiate key material rotation for this key. You can initiate on-demand key rotation a maximum of 25 times.

Number of on-demand rotations remaining

25 rotations available

Key materials (1)

Filter key materials

Key material ID

Date

Key material state

Rotation type

52a17f1083a3f7ca5fc2cafa63e97671...

-

Current

-

Amazon S3

Buckets

Create bucket

General configuration

AWS Region

US East (N. Virginia) us-east-1

Bucket type

General purpose

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

Directory

Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace

Choose the namespace where you want to create your bucket.

Global namespace

By default, S3 creates general purpose buckets in the global namespace.

Account Regional namespace (recommended)

General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name

devops-kms-lab-asif-2026

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-).

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Choose bucket

Format: s3://bucket/prefix

Amazon S3

Buckets

Create bucket

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type

Server-side encryption with Amazon S3 managed keys (SSE-S3)

Server-side encryption with AWS Key Management Service keys (SSE-KMS)

Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

AWS KMS key

Choose from your AWS KMS keys

Enter AWS KMS key ARN

Available AWS KMS keys

arn:aws:kms:us-east-1:331116047691:key/7827334e-c308-455d-aa3c-39c86643efa1

Create a KMS key

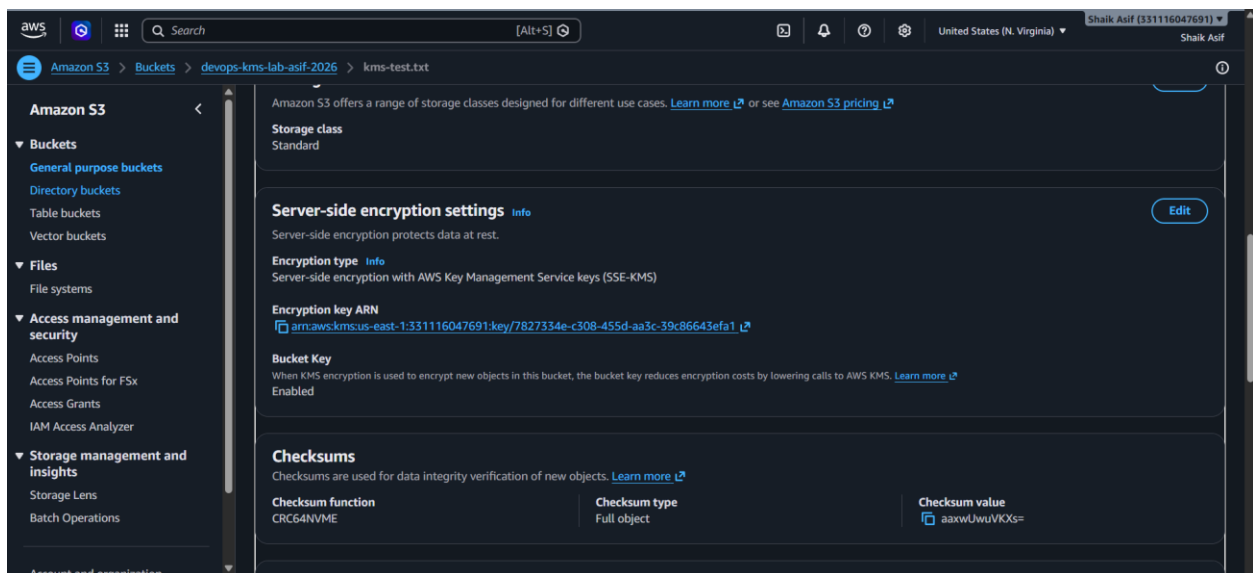
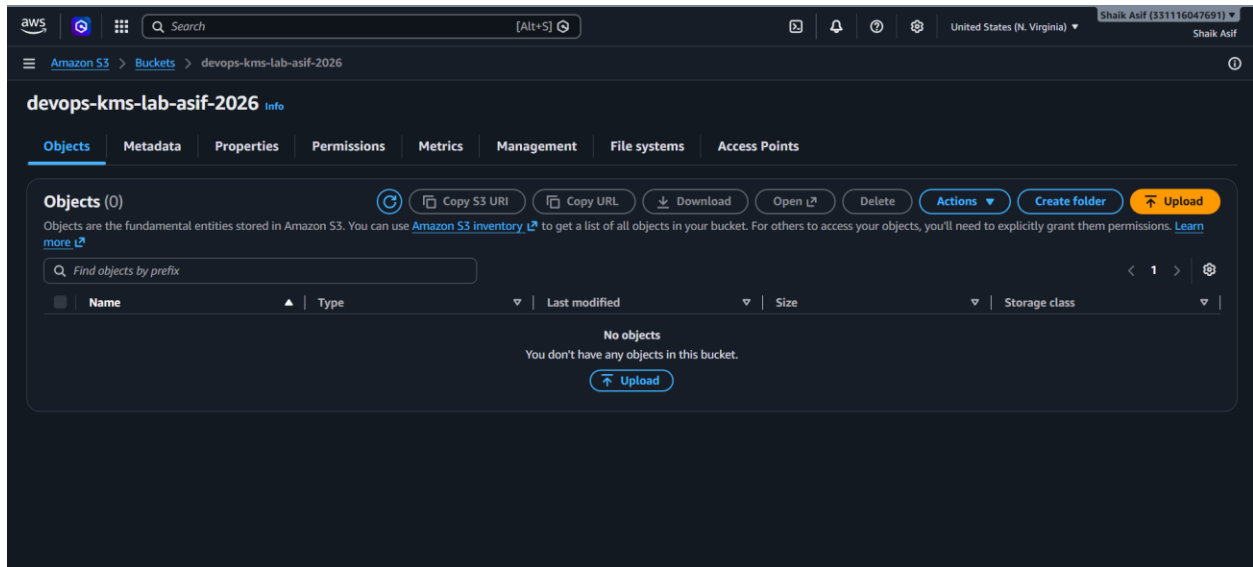
Bucket Key

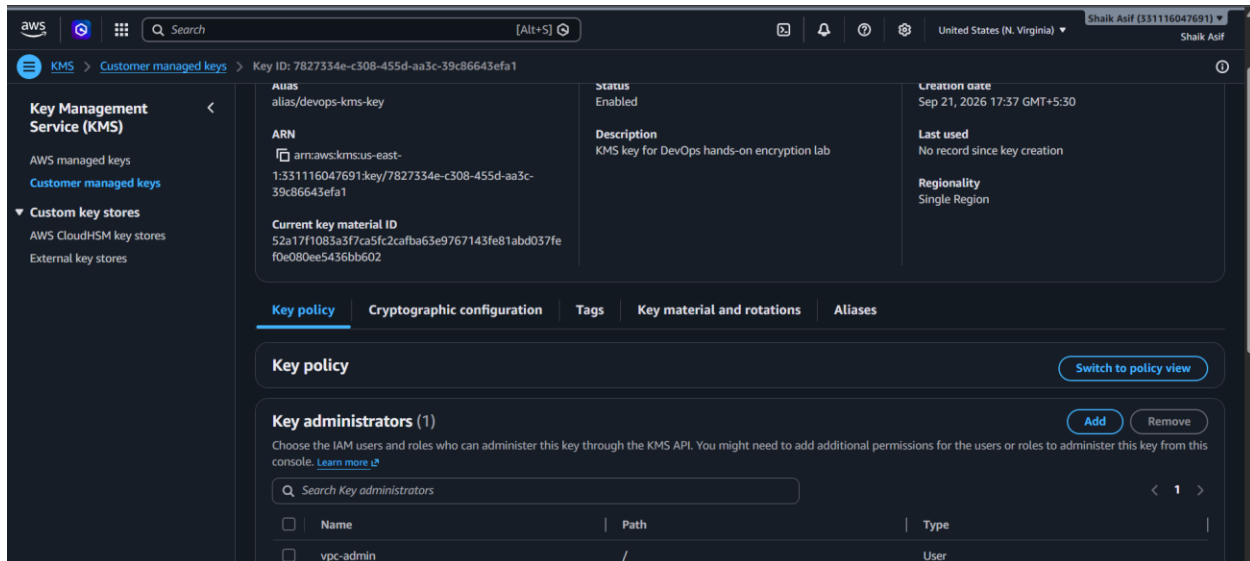
Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS.

Disable

Enable

Advanced settings





```
asifs@Asif_Shaik MINGW64 ~/downloads (master)
$ aws sts get-caller-identity
{
  "UserId": "AIDAU2GAN5VFS33NMUWU7",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/vpc-admin"
}

asifs@Asif_Shaik MINGW64 ~/downloads (master)
$ aws kms describe-key \
> --key-id 7827334e-c308-455d-aa3c-39c86643efa1 \
> --region us-east-1
{
  "KeyMetadata": {
    "AWSAccountId": "331116047691",
    "KeyId": "7827334e-c308-455d-aa3c-39c86643efa1",
    "Arn": "arn:aws:kms:us-east-1:331116047691:key/7827334e-c308-455d-aa3c-39c86643efa1",
    "CreationDate": "2026-09-21T17:37:02.015000+05:30",
    "Enabled": true,
    "Description": "KMS key for DevOps hands-on encryption lab",
    "KeyUsage": "ENCRYPT_DECRYPT",
    "KeyState": "Enabled",
    "Origin": "AWS_KMS",
    "KeyManager": "CUSTOMER",
    "CustomerMasterKeySpec": "SYMMETRIC_DEFAULT",
    "KeySpec": "SYMMETRIC_DEFAULT",
    "EncryptionAlgorithms": [
      "SYMMETRIC_DEFAULT"
    ],
    "MultiRegion": false,
    "CurrentKeyMaterialId": "52a17f1083a3f7ca5fc2ca6ba63e9767143fe81abd037fef0e080ee5436bb602"
  }
}
```

```
asifs@Asif_Shaik MINGW64 ~/downloads (master)
$ aws kms get-key-rotation-status \
> --key-id 7827334e-c308-455d-aa3c-39c86643efa1 \
> --region us-east-1
{
  "KeyRotationEnabled": true,
  "KeyId": "arn:aws:kms:us-east-1:331116047691:key/7827334e-c308-455d-aa3c-39c86643efa1",
  "RotationPeriodInDays": 365,
  "NextRotationDate": "2027-09-21T17:41:50.131000+05:30"
}
```

Observations

1. A customer-managed symmetric KMS key was successfully created.
2. The key was configured for **Encrypt and decrypt** operations.
3. The alias alias/devops-kms-key was created for easier identification.

4. The KMS key remained in the **Enabled** state.
5. **Automatic key rotation** was successfully enabled.
6. The rotation period was confirmed through CLI as **365 days**.
7. vpc-admin was configured as both a **Key Administrator** and **Key User**.
8. An S3 bucket was configured with **SSE-KMS** encryption.
9. S3 displayed the ARN of the customer-managed KMS key being used for encryption.
10. **S3 Bucket Key** was enabled to reduce KMS request overhead.
11. CLI verification confirmed that the KMS key is customer-managed, symmetric, enabled, and configured for encryption/decryption.
12. The kms:ListAliases CLI command returned AccessDeniedException, demonstrating that **KMS key permissions and IAM identity permissions are separate authorization considerations**.

Result / Conclusion

AWS KMS was successfully implemented using the AWS Management Console. A customer-managed symmetric KMS key was created and configured for encryption and decryption. An alias was created, automatic key rotation was enabled, and key administration and usage permissions were configured. Amazon S3 was integrated with the KMS key using SSE-KMS, with S3 Bucket Key enabled. The implementation was further verified through AWS CLI, confirming the key state, key type, key usage, customer-managed status, and automatic rotation configuration.